

BENCHMARKS.md

Benchmarks — measured, not estimated

Every number in this file was produced on one machine and says how it was produced. Where a figure is old, the date is stated; where something was **not** measured, it says so instead of guessing.

Hardware of record: Windows 11 · AMD Ryzen 5 5600G (6 cores / 12 threads, **no** CUDA GPU) · 15.8 GB RAM · 85 GB free disk. Local inference runs through **Ollama** on `127.0.0.1:11434`, all models at **Q4_K_M** quantisation.

1. Local model throughput (measured Sep 14 2026)

| Model | Params | On disk | Prefill | Decode | Notes |

|---|---|---|---|---|

| `granite3.3:2b` | 2.5B | 1.55 GB | **1,375 tok/s** | **113-124 tok/s** | Fastest by far. On some summaries it switches to English — acceptable for drafts, not for output |

| `qwen3:4b-q4_K_M` *(retirado del set local el 15/09/2026)* | 4B | 2.6 GB | 450 tok/s | 90-125 tok/s | Hybrid reasoning model: with a small token budget it returns an **empty string** (all budget spent thinking). Must be called with `"think": false` or a large budget. Se retiró por no tener capacidad única (tools+thinking ya están en el 7B, tools en el 2B), por medir peor que el 2B en el contrato estricto (2/3) y por ser el que cruzó el precipicio de VRAM (7,6 GB → 331 s de TTFT a 16k). Las mediciones se conservan por trazabilidad |

| `deepseek-r1:7b` | 7.6B | 4.7 GB | 461 tok/s | 63-67 tok/s | Best local quality in the engine planilla (\$2) |

| `qwen2.5vl:7b` | 8.3B | 6.0 GB | 250 tok/s | 70 tok/s | Vision model (reads images attached to nodes) |

Method: `POST /api/generate` with a fixed prompt, `num_predict` 400-500, `temperature` 0, after a warm-up call so the model is resident. Prefill read from `prompt_eval_duration`, decode from `eval_duration`, and the decode rate cross-checked against wall-clock time.

Not viable on this machine: `gpt-oss:20b` (14 GB) and `qwen3:30b` (19 GB) do not fit in 15.8 GB of RAM. That is a hardware limit, not a preference.

Why this matters for the router. Prefill is the bottleneck, not decode: with a 12.5k-token prompt (the agent context measured on this box) the times to first token are ~9 s for `granite3.3:2b` and ~27-28 s for the 4B/7B models. That is the measured reason the router sends **drafting** to the 2B and reserves the bigger models for tasks where depth actually pays.

2. Engine planilla — both motors on the same real tasks

Source: [`EVALUACION.md`](EVALUACION.md). Each engine runs **the canvas's real tasks** through the same path the app uses (its spec, its validation, its cache — skipped on purpose — and its cost accounting). The verdict comes from **code**, not from a model judging another model: did the plan work, how long did it take, what did it cost?

Engine	Passed	Mean time	Tokens	Cost
deepseek-r1:7b	4/5	15.2 s	9,436	US\$0
granite3.3:2b	3/5	4.3 s	8,308	US\$0

Test	deepseek-r1:7b	granite3.3:2b	What it measures
voz-enfocar	pass · 22.4 s	pass · 9.4 s	understands a cleanup order (a single `enfocar`)
voz-crear	pass · 16.2 s	fail · 2.6 s	adds new ideas without destroying the canvas
voz-delegar	fail · 13.1 s	fail · 2.2 s	recognises when the deep engine should take over
condensar	pass · 13.0 s	pass · 2.3 s	condenses respecting the contract (title, description, match 0..1)
braindump	pass · 11.2 s	pass · 5.1 s	breaks one idea into a map with a root and branches

Canvas used for the run: 15 nodes. One engine at a time, unloaded from memory at the end. This is the kind of measurement that changes design decisions — `voz-delegar` failing on both models is exactly why escalation is decided by the router's declared plan instead of asking the small model its opinion.

3. Cache effect (read off the app in normal use)

`GET /api/ai/cache` on the live graph (Sep 15 2026):

Counter	Value
Entries	80 (cap 300, `CACHE_VER 3`)
Exact hits	2
Semantic hits	2
Misses	130
Tokens avoided — exact tier	3,488
Tokens avoided — semantic tier	6,555
Total tokens avoided	10,043

Earlier milestone measurement of the same mechanism: an identical repeat of one expert run went from

27.9 s / 9,061 tokens → 0.4 s / 0 tokens.

How the two tiers work (`costo.rs` + `semantica.rs`):

- **Exact tier** — key = node + prompt + provider + schema + `CACHE\_VER`. The key deliberately carries

- no clock: the first implementation hashed a prompt containing `now\_iso()` and scored 6 misses / 0 hits on identical runs.

- **Semantic tier** — compares by meaning with cosine similarity over Gemini embeddings

(`gemini-embedding-001`, 3072 dimensions), threshold **0.92**. Without a key, without network, or if the embedding call fails once, it falls back to a dependency-free normalised-token comparison (containment, with a length guard for short prompts) and disables the vector path for that run instead of retrying.

- **Cost of the cache miss never disappears silently:** a model without a declared rate reports `unknown`, **not** `\$0` — "no rate declared" and "free" are different facts.

4. T0→T1 — the app times its own value

From the raw brain dump (T0) to the first AI proposal **approved by a human** (T1), recorded by the app itself (`GET /api/metrics`):

Conversions	Deltas (minutes)	Median
5	3.8 · 24.7 · 24.8 · 61.6 (+1 same-session at 0.0)	24.7

This is the metric the product is steered by: not "how much did the model generate", but how long it took for generated work to become something a human accepted.

5. Runtime footprint

Measurement	Value	How
Native shell RSS	(`app.exe`, v0.3.4, 55 nodes loaded) 38.9 MB	`Get-Process app` (Sep 15 2026)

Reading it honestly: this is the **native shell only**. The WebView2 child processes that render the canvas are **not** attributed here, so it is not the total desktop footprint — it is the part the Rust side owns. Reported this way on purpose.

6. What was NOT measured (and is therefore not claimed)

- Total desktop footprint including all WebView2 renderer children.
- macOS or Linux behaviour — there are no builds for those platforms yet.
- GPU inference: this machine has no CUDA GPU, so the local path is CPU-only.
- Peak memory while synthesising audio (Kokoro TTS) — the probe captured the virtualenv launcher, not the interpreter child, so the figure is pending rather than invented.

*Related: EVALUACION.md (engine planilla, in Spanish),
../README.md
(measured numbers summary), [adr/](adr) (why these decisions were made).*

CI_RESULTS.md

Resultados de CI

Paso	Resultado
Lint (TS)	■ 0 errores
Build (Vite)	■ 0 warnings
Rust tests	■ 294 passed
Demo log (10 turnos)	■ `voz(ui).turno.cerrado = 10`

CREDITS.md

Créditos

- **Autor**: Tomas Pieruz (NeonEden) – arquitectura, React, Rust, Tauri.
- **Dependencias clave**: React 19, Vite 6, Tauri 2, Lucide[■]react, pypdf, reportlab, pdfplumber, rust[■]serde, serde_json.
- **Agradecimientos**: comunidad Open[■]Source, proveedores DeepSeek, Azure, AssemblyAI, Speechmatics, GitHub.

DEVLOG.md

Bitácora de desarrollo

Generado desde el historial de git (Conventional Commits). No se edita a mano.

`96` commits · rama `main` · actualizado 2026-09-14 14:09

Últimos 14 días:

2026-09-14 · 16 commit(s) · +1503 –121

****Nuevas capacidades****

- taxonomia geometrica de 5 estados + busqueda Tavily + tags del nucleo limpios

****Correcciones****

- proponer una sola vez por corrida (y limpiar el ruido de «Actualizar»)
- los hallazgos entran como propuestas del agente, no directo al lienzo
- la investigación se ve y se aplica sin depender del panel de voz
- rechazar ids de motor que no existen
- el saneo tambien limpia aristas repetidas, no solo colgadas
- luz de borde en las formas recortadas

****Rendimiento****

- el prompt se ordena para el cache de prefijo y la app mide el ahorro

****Documentación****

- las cuatro capas del mapa del proyecto y la regla de sincronizacion con la evidencia
- Tavily verificado (la clave responde con el texto completo de las paginas) y tutorial corregido
- como funciona NodeFlow de punta a punta, con la tabla honesta de limites

****Infraestructura****

- v0.3.0

****Guardado automático****

- 4 punto(s) de seguridad sin mensaje propio (omitidos de la bitácora)

2026-09-13 · 62 commit(s) · +11025 –6343

****Nuevas capacidades****

- el nodo que crece por fases (semilla, fricción, capsula, hexagono)
- acción actualizar (mutar un nodo que ya existe) y tolerancia a la clave mal pegada
- una conversación en curso manda el pedido a un modelo a la altura
- hilo de conversación — de comandos sueltos a conversación
- motor por tipo de tarea, con lo local como camino principal
- NodeFlow puede delegar en Hermes y usar sus herramientas
- bucle hablado — Kokoro local y voz selectiva decidida en código
- Fase A — Norte Estratégico + macro-nodos con linaje (poda sin pérdida)
- automatic profile learning, honest wording and readable panel
- global engine selector - one place decides where the AI runs
- per-task local/cloud mode with measured cost and cache trace

****Correcciones****

- el ganador por prueba comparaba ms contra tokens
- medir al modelo real y no a la cache
- tags duplicados y contraste de texto, con auditor
- el comando 'enfocar' enfoca de verdad (y no colapsa el lienzo por sorpresa)
- the automatic pass uses the selected engine, not a cloud key
- the panel footer no longer claims an Express backend
- light form fields in the add-API panel and a creator credit
- make local engines usable, messages honest and engines honest too
- retry the API bind instead of opening a window without API

****Reorganización****

- one place per action - toolbar reorganized into clusters and menus

****Documentación****

- planilla medida y sus conclusiones
- notas de v0.1.1 con el Motor de Condensación y los SHA-256
- bitácora de la Fase A
- notas de v0.1.0 listas para publicar
- add README (EN/ES), roadmap, ADRs, MIT license and real env example

****Infraestructura****

- v0.2.0
- limpieza automática de target/debug (los tests lo inundaban)
- fuera el backend Express del prototipo (server.ts)
- limpieza de Dependabot
- v0.1.1
- unignore src/data, sync lock and add CI
- merge the v0.1 native rewrite onto the NeuralMind prototype history
- fix identity - nodeflow-desktop 0.1.0, MIT

****Guardado automático****

- 24 punto(s) de seguridad sin mensaje propio (omitidos de la bitácora)

2026-09-12 · 15 commit(s) · +37588 –500

2026-09-11 · 1 commit(s) · +1781 –216

****Nuevas capacidades****

- add rotatable idea template packs

2026-09-09 · 2 commit(s) · +10558 –11

****Nuevas capacidades****

- initialize NeuralMind interactive node map

EVALUACION.md

Planilla de evaluación — medir en vez de suponer

Cada motor corre **las tareas reales del lienzo** por el mismo camino que usa la app (su spec, su validación, su caché —salteada a propósito— y su contabilidad de costo). El veredicto lo da **código**, no la opinión de un modelo sobre otro: ¿el plan sirvió, cuánto tardó, cuánto costó?

- Lienzo de la corrida: **15 nodos**.
- Cinco pruebas: ``voz-enfocar``, ``voz-crear``, ``voz-delegar``, ``condensar``, ``braindump``.
- Todo **local** y **US\$0**: no gasta plan de tokens. Un motor por vez, y se descarga de la VRAM al terminar.

Motor	Aciertos	Tiempo medio	Tokens	Costo
<code>`deepseek-r1:7b`</code>	4/5	15.2 s	9436	US\$0
<code>`granite3.3:2b`</code>	3/5	4.3 s	8308	US\$0

Prueba por prueba

Prueba	deepseek-r1:7b	granite3.3:2b	Qué mide
<code>`voz-enfocar`</code>	✓ 22.4 s ✓ 9.4 s		entiende una orden de limpieza (un solo <code>`enfocar`</code>)
<code>`voz-crear`</code>	✓ 16.2 s ✗ 2.6 s		suma ideas nuevas sin destruir el lienzo
<code>`voz-delegar`</code>	✗ 13.1 s ✗ 2.2 s		reconoce cuándo hay que ir al motor profundo
<code>`condensar`</code>	✓ 13.0 s ✓ 2.3 s		condensa respetando el contrato (título, descripción, match 0..1)
<code>`braindump`</code>	✓ 11.2 s ✓ 5.1 s		descompone una idea en un mapa con raíz y ramas

Qué dice esto (y qué no)

- **El modelo chico gana el bucle rápido**: ``granite3.3:2b`` promedia **4,3 s** contra **15,2 s** del R1, y en tres de las cinco pruebas es el único que llega en tiempo razonable. Para hablar con el lienzo, el local chico alcanza.
- **Donde el chico falla, el grande sirve**: ``voz-crear`` (sumar ideas nuevas) lo resolvió sólo ``deepseek-r1:7b`` — 2 nodos con título. Es el candidato natural para "pensar despacio".
- **Ningún modelo local reconoce cuándo delegar** (0 comandos en los dos, tres corridas seguidas). Esa decisión necesita el motor de nube: es evidencia, no corazonada.
- **Costo real de operar el lienzo en tu máquina**: **US\$0** en las diez corridas.

Límites honestos: cinco pruebas dan una **señal**, no un veredicto; el veredicto es binario (¿cumple el contrato?) y no mide calidad subjetiva. Y las pruebas que fallan en los dos motores dicen tanto del límite del modelo como del de la tarea.

FEATURES.md

Funcionalidades clave

- **Voice-first UI** – push to talk con reuso de sesión (90 s timeout).
- **Agentes IA integrados** – DeepSeek, OpenAI, Azure; selección dinámica.
- **Persistencia local segura** – claves API guardadas en `localStorage`.
- **Auto merge** – GitHub Actions verifican TSC, build y tests antes de mergear.
- **Demo Vercel** – despliegue listo to play.

FLUJO.md

Flujo de trabajo

La regla es simple: **el trabajo se guarda solo y cada versión es un punto al que se puede volver.**

Guardado automático

```
```bash
scripts/checkpoint.sh # una vez: verifica (tsc + tests de Rust) y guarda
scripts/checkpoint.sh --watch # lo repite cada 5 minutos mientras trabajás
```
```

Si no hay cambios, no hace nada. Si hay, **primero corre los chequeos** y sólo si pasan commitea y empuja. Si algo falla, no guarda y deja el motivo en `.git/checkpoint.log`: nunca se sube algo roto.

En esta máquina corre además una tarea programada de Windows (`NodeFlow Checkpoint`) que lo ejecuta cada 5 minutos, así que no hace falta acordarse de nada.

Bitácora (lo que se muestra)

```
```bash
scripts/devlog.sh 14 # arma los últimos 14 días
```
```

Se genera **desde el historial de git**, no se escribe a mano, y sale en dos lugares:

- `docs/DEVLOG.md` — el registro del repositorio.
- `/NodeFlow/devlog/AAAA-MM-DD.md` — la nota del día en Obsidian, con frontmatter.

Nueva versión

```
```bash
scripts/release.sh --dry-run # ver qué versión saldría
scripts/release.sh patch # sube patch|minor|major, compila instaladores, etiqueta y sube
```
```

Cada versión queda como **tag anotado** (`v0.1.0`, `v0.2.0`, ...): es el punto exacto al que se vuelve si algo sale mal (`git checkout v0.1.0`).

Respaldo

```
```bash
scripts/backup.sh # bundle completo del repositorio en ../backups (rota los últimos 10)
```
```

Disciplina de commits

Conventional Commits (`feat`, `fix`, `refactor`, `docs`, `test`, `chore`) y SemVer en las versiones. El mensaje explica **qué** y **por qué**; los números medidos van en el cuerpo.

Red de seguridad completa

| Capa | Qué protege |
|---|---|
| Checkpoint cada 5 min | No se pierde trabajo entre sesiones |
| Chequeos antes de guardar | No entra código que no compila ni rompe tests |
| CI en cada push (GitHub Actions) | `npm ci` + `tsc` + `cargo test` en un entorno limpio |
| Tags por versión | Punto de retorno exacto |
| Bundle de respaldo | El historial completo, fuera del repositorio |
| Backups rotativos de la app | El lienzo y las propuestas (`.nodeflow/backup-\*.json`) |

Guardado automático de fondo (sin ventanas)

Una tarea de Windows llamada **NodeFlow Checkpoint** corre cada **10 minutos**:

```
```
wscript.exe //B "scripts\auto-oculto.vbs" → bash scripts/auto.sh
```
```

- El lanzador `.vbs` existe por una razón concreta: llamar a `bash.exe` directamente **abría una consola** cada vez. `WScript.Shell.Run`` con estilo de ventana `0`` la mantiene oculta.
- `scripts/auto.sh`` = respaldo diario (si corresponde) + punto de guardado.
- **Sólo chequea lo que cambió**: si tocaste `.ts/.tsx/.json/.css`` corre `tsc``; si tocaste `.rs/.toml`` corre los tests de Rust; si sólo hay docs, no corre nada. El mensaje del commit dice qué se verificó.
- Sin cambios → no hace nada (sólo deja la marca en `.git/checkpoint.ultima``).

Comandos a mano:

```
```bash
bash scripts/checkpoint.sh # verificar y guardar ahora
bash scripts/checkpoint.sh --watch # modo vigilante en primer plano
schtasks -query -tn "NodeFlow Checkpoint" # ver la tarea
schtasks -end -tn "NodeFlow Checkpoint" # pausar el automático
schtasks -run -tn "NodeFlow Checkpoint" # dispararlo ya
```
```

GETTING_STARTED.md

Guía de inicio rápido

```
```bash
```

### Clonar

```
git clone https://github.com/NeonEden/NodeFlow.git
cd NodeFlow/nodeflow-desktop
```

### Instalar dependencias

```
pnpm install
```

### Ejecutar demo local

```
pnpm dev # abre http://localhost:5173
```

### Probar voz

- pulsa **Ctrl+Alt+Space** para activar el micrófono.
- después de hablar, el turno se cierra y el log muestra ``voz(ui).turno.cerrado``.

### Publicar demo en Vercel (solo push)

```
git push origin main # Vercel detecta el `vercel.json` y despliega.
```
```

MAPA-DEL-PROYECTO.md

Mapa del proyecto NodeFlow

> **Qué es esto**: la estructura con la que el lienzo de NodeFlow refleja el estado real del proyecto
> y permite dirigir el progreso. No es documentación *sobre* la app: es el esquema *del* lienzo.

Las cuatro capas

| Capa | Qué guarda | Cómo se reconoce |
|---|---|---|
| **N0 · Norte** (1 nodo) | La tesis y el objetivo | Único nodo raíz real de todo el grafo |
| **N1 · Pilares** (8) | Las capacidades que perduran | Categorías estables: NÚCLEO · DATOS · UX · ARQUITECTURA · GOBERNANZA · SEGURIDAD · SISTEMAS · VISIÓN |
| **N2 · Capacidades** | Fases y features concretas | La madurez dice cuánta evidencia tienen |
| **N3 · Evidencia** | Investigaciones y fuentes | Categorías INVESTIGACIÓN y FUENTE, colgando de la capacidad que alimentan |
| **N4 · Movimientos** | **Lo que dirige** | PENDIENTE · DECISIÓN · RIESGO · HITO |

La madurez es evidencia, no entusiasmo

| Nivel | Forma | Significa |
|---|---|---|
| 1 | ■ Semilla | Dicho, sin contrastar |
| 2 | ■■ Fricción | Contrastado: hay una fuente, un dato o una prueba externa |
| 3 | ■ Cápsula | Probado: funciona en una corrida real |
| 4 | ■ Axioma | Sostenido: verificado y en uso |
| 5 | ■ Hexágono | Artefacto: entregado o publicado |

Las aristas tienen semántica

`contiene` (Norte → pilar) · **`requiere`** (capacidad → capacidad: la dependencia) ·
`bloquea` (riesgo → capacidad) · `entrega` (capacidad → hito) ·
`alimenta` (fuente → investigación → capacidad) · `decide` (decisión → capacidad)

Con eso, «¿qué sigue?» deja de ser intuición: es el nodo con más **`requiere`** sin cumplir y más **`bloquea`** encima. Eso es el camino crítico.

La regla que lo mantiene vivo

El mapa se genera de la evidencia; no se mantiene a mano. Igual que la bitácora sale de git (`scripts/devlog.sh`), un `scripts/mapa.py` lee:

- `git log` (qué se cerró, con qué mensaje) · las releases publicadas
- `docs/\*.md` (el estado declarado) · el skill del proyecto (los pendientes elegidos)
- la cola de propuestas y los artefactos (`evaluacion.json`, `investigacion.json`)

...y emite **propuestas** (`POST /api/graph/node` / `/api/graph/edge`) que el humano aprueba. Corre en

el mismo checkpoint de 5 minutos que ya existe: así el lienzo nunca queda viejo y el estado actual es una **consulta**, no un recuerdo.

Estado al 14/09/2026 (lo que el mapa dice hoy)

- **Columna vertebral**: 15 nodos con categoría y madurez reales.
- **Escombros**: 11 nodos de un lote socrático/telemetría, sin madurez y sin dueño — se reenganchan a la capacidad que preguntan (telemetría, provenance) y se recategorizan como ANÁLISIS.
- **Movimientos sembrados**: 6 pendientes, 2 riesgos, 2 decisiones, 2 hitos.
- **Dependencias**: 5 aristas `requiere` reales.

PLAN-CEREBRO-RESIDENTE.md

Plan · El cerebro residente: que el modelo habite el campo de los nodos

Objetivo (en palabras del usuario): que Hermes *habite* el lienzo. Que la memoria, el contexto y la visión del proyecto *crezcan desde adentro de la app* —no desde el chat— y que, cuando haya que modificar

algo, se haga *desde adentro de la app*, con las herramientas de Hermes ejecutando ahí.

Este documento es la evaluación de viabilidad + el plan. Todo lo marcado *verificado* se probó el 15/09 sobre la máquina, no se supuso.

0. Estado del plan (actualizado 15/09, noche)

| Fase | Qué | Estado |

|---|---|---|

| 0 | Habilitar las 22 tools MCP del lienzo para Hermes | ■ **hecho y permanente** (``exclude: []``; verificado en un proceso nuevo: contó los nodos del lienzo con sus tools) |

| 1 | Corte vertical: ``delegar`` con sesión nombrada + nota episódica | ■ **hecho y verificado en vivo**: dos turnos en la misma sesión (prueba dura en ``state.db``), notas en ``/cerebro/`` con hora local |

| 2 | El contexto lo arma la app (recuerdo dirigido), no el lienzo masticado | ■ **hecho**: ``cerebro::prompt_turno`` (visión + top-6 BM25 + foco + contadores + puntero a las herramientas); la investigación comparte la misma sesión; el contexto usado se guarda en ``delegacion.json`` |

| 3 | Panel «Pensar desde el lienzo» (pedido, progreso, aterrizaje, contexto usado) | ■ pendiente |

| 4 | De ``hermes -z`` a ``hermes serve``: delegar → habitar | ■ pendiente |

Extra hecho esa noche, por directiva del usuario (*«las investigaciones tienen que aportar información que*

sirva para la construcción del cerebro/lienzo»): la investigación aterriza en *un* nodo con la evidencia

adentro de su nota y colgado del Norte (antes dejaba 5-6 nodos de bibliografía suelta), y la síntesis tiene

fallback declarado (DeepSeek → Hermes) con el error visible en vez de morir en silencio.

1. Dónde estamos hoy (estado real, verificado)

El puente existe, pero en un solo sentido y sin memoria

- **NodeFlow** → **Hermes**: `POST /api/ai/delegar`` corre `hermes -z ""`` como subproceso sin consola (`voz::hermes_exe()`, tope 240 s, `NODEFLOW_HERMES`` para mover el binario). **Es one-shot y sin continuidad**: cada delegación arranca sin recordar la anterior. Medido: una delegación real tardó 215 s.

- **Hermes** → **NodeFlow**: el servidor MCP propio (`mcp-server/nodeflow_mcp.py``, 22 herramientas) expone el lienzo, la bóveda, el jardín y los expertos. **Hoy está apagado** para Hermes por `mcp_servers.nodeflow.tools.exclude: [*]`` en `config.yaml`` — por eso esta sesión tuvo que hablarle por stdio (y funcionó: así se creó y aprobó el nodo de rendimiento).

Lo que la app ya tiene y sirve de base (no hay que inventarlo)

| Pieza | Estado |
 |---|---|
 | Cola de propuestas + HITL con auditoría | En producción: nada toca el lienzo sin aprobación; `hermes`` visible |
 | Hilo de diálogo con foco | 12 turnos (se muestran 6), expira a los 30 min, vive en `./nodeflow/dialogo.json`` |
 | Memoria de la bóveda | BM25 sobre todas las notas, con `limite`` acotado a 1..40 |
 | Memoria continua del proyecto | `GET /api/vault/memory`` |
 | Investigación por fases · expertos · borradores locales | En producción |
 | Guardas de escritura | `base_revision`` + rescate de escrituras del agente + reconciliación |
 | Auto-actualización firmada | v0.3.5 publicada el 15/09 |

Las puertas de Hermes (probadas hoy)

| Puerta | Qué es | Veredicto |
 |---|---|---|
 | `hermes -z ""`` | one-shot | **Sin estado.** Es lo que usa la app hoy |
 | `hermes --resume -z ""`` | reanuda una sesión **por id** | **Funciona**: reanudó una sesión real y respondió con su contenido |
 | `hermes -c -z ""`` | reanuda **por título** | Funciona si la sesión ya existe (`hermes sessions rename``). `--create-if-missing`` **no existe** (el propio mensaje de error lo sugiere y es falso) |
 | `hermes serve`` | backend JSON-RPC/WebSocket (default `127.0.0.1:9119``), headless: **es el que usa la app de escritorio** | La puerta para ser una *superficie* de verdad: streaming, eventos de herramienta, aprobaciones |
 | `hermes acp`` | Agent Client Protocol (editores: VS Code / Zed / JetBrains) | Alternativa estándar, pero orientada a IDE: la app necesitaría un cliente ACP |
 | `hermes mcp serve`` | Hermes **como servidor MCP** | **Ojo, corrección:** expone 10 herramientas de **conversaciones** (`conversations_list``, `messages_read``, `messages_send``, `events_poll/wait``, `permissions_``) — es un puente de mensajería, **no** ejecuta turnos del agente. No sirve para "que el modelo trabaje desde la app" |

2. La arquitectura propuesta: tres capas, una sola verdad

****Fase 1 · El corte vertical: memoria que sobrevive (medio día).**** En el backend, `delegar` deja de ser one-shot: guarda un nombre de sesión por proyecto (`nf-cerebro`) en `.nodeflow/` y llama `hermes -c nf-cerebro -z ...`; al terminar, escribe el intercambio como ****nota episódica fechada**** en la bóveda

(con frontmatter: `tipo: turno`, `pedido`, `resultado`, `sesion`). Criterio: cerrar la app, reabrir, pedir "seguí con lo de ayer" y que el pedido resuelva con el historial real. Sin la app abierta, `hermes -c nf-cerebro` también debe poder continuar el hilo (probado que la continuidad por id/título funciona).

****Fase 2 · El contexto lo arma la app (1-2 días).**** El prompt delegado se compone desde la bóveda: top-k BM25 + foco del hilo + nodos vecinos + pendientes. Se expone en el panel para poder ****ver qué se le mandó****

(transparencia: hoy el prompt es invisible). Criterio: pedir algo que requiera un nodo viejo y ver que lo trae sin nombrarlo.

****Fase 3 · La boca en la app (2-3 días).**** Panel «Pensar desde el lienzo»: se escribe el pedido ahí, se ve el

progreso (el mismo polling que ya usa `delegar`/investigación), y el resultado aterriza como nodos propuestos +

nota. Nada sale del lienzo. Criterio: un pedido real resuelto de punta a punta sin abrir esta chat.

****Fase 4 · De delegación a superficie (1-2 semanas, el salto grande).**** Pasar de `hermes -z` a `hermes serve`

(JSON-RPC/WebSocket, `127.0.0.1:9119`): la app se conecta como cliente, como lo hace la app de escritorio de

Hermes. Se gana: streaming token a token, eventos de cada herramienta en vivo, aprobaciones nativas, turnos

largos sin el tope de 240 s, y multi-turno sin re-lanzar procesos. Criterio: un turno con 5 herramientas mostrando cada paso en el panel, y el registro quedando en la bóveda.

Esta es la fase que convierte "delegar" en "habitar". Las anteriores son útiles sin ella.

****Fase 5 · La memoria del proyecto, derivada (continuo).**** Cron de la app que, cada N horas o al cerrar una

sesión, sintetiza: «visión del proyecto» (nodo del Norte), decisiones tomadas, preguntas abiertas y deuda

técnica — regenerado ****desde el grafo****, nunca escrito a mano. Las `references/` de los skills de Hermes pasan

a ser un resumen derivado de la bóveda (Hermes recuerda **cómo**, la bóveda **qué**).

4. Viabilidad: veredicto honesto

****Sí, es viable****, y no requiere investigación nueva: las tres cuartas partes están construidas (cola HITL,

bóveda como verdad, BM25, hilo, guardas, MCP en los dos sentidos). Lo que falta es ****cableado y política****,

no tecnología. Esfuerzo realista: ****Fases 0-3 en 3-5 días de trabajo****; la Fase 4 es la inversión grande (1-2 semanas) y es la única que cambia la naturaleza del sistema.

Riesgos y límites (los que pueden hacer fracasar esto)

1. ****El tope de 240 s y el one-shot**** son la jaula actual: mientras ``delegar`` sea un subproceso, el "habitante" es un visitante. Fase 1-3 lo hacen **útil**; Fase 4 lo hace **residente**.
2. ****Costo y ruido por turno:**** un turno con herramientas es 10-50× un pedido de lienzo. Sin la regla 3 (contexto recuperado, no completo) el proyecto se vuelve caro justo cuando crece.
3. ****La placa:**** el "cerebro local permanente" (meta del usuario) no convive con ventanas de 16k: medido, 7B@16k = 426 s de TTFT. El diseño honesto es local para lo chico del bucle y nube para lo profundo, con el router que ya existe (``auto:tarea``).
4. ****Proveedor inestable:**** hoy DeepSeek no estuvo fino (errores de conexión y respuestas pobres). Cualquier diseño residente necesita ****fallback declarado**** (Hermes ya tiene ``hermes fallback``): si el residente depende de un proveedor que falla, la app parece rota.
5. ****Fricción de aprobación:**** si todo es propuesta, el agente parece inútil; si nada lo es, el usuario pierde el control. Ya existe el perfil de autonomía aprendido (``/api/hitl/*``): la política de qué se auto-aprueba es una ****decisión de producto****, no un detalle técnico.
6. ****Doble escritor:**** ya resuelto por ``base_revision`` + rescate + propuestas, pero hay que ****no romperlo****: todo camino nuevo de escritura pasa por el mismo contrato.

5. Decisiones abiertas (para el usuario)

1. ****¿Habilitamos las 22 herramientas MCP**** del lienzo para Hermes (Fase 0)? Es la que más valor da por minuto invertido y es reversible en un comando.
2. ****¿Qué se auto-aprueba?**** Propuesta conservadora: el agente puede ****crear notas y nodos nuevos**** sin pedir, pero ****modificar o borrar**** lo existente siempre pide aprobación.
3. ****¿Fase 4 sí o sí, o alcanza con Fases 1-3?**** Las 1-3 dan un cerebro con memoria que vive en el lienzo. La 4 agrega ver al agente trabajar en vivo y conversaciones largas de verdad.

Borrador del 15/09/2026. Basado en lo verificado en esta sesión: continuidad por id/título (probada), ``--create-if-missing`` inexistente (probado), ``hermes mcp serve`` = conversaciones y no ejecución (probado), ``hermes serve`` = JSON-RPC/WebSocket :9119 (documentado en su ``--help``), y los números de escala en ``references/rendimiento-y-escala.md`` del skill ``nodeflow-mcp``.

PLAN-INTEGRACION-CEREBRO.md

Plan · Incorporar el cerebro a NodeFlow (Fase 5 en adelante)

Pedido del usuario (16/09): «ya está el cableado casi completo, pero falta el siguiente paso: incorporarte en NodeFlow, darte espacio para que crees tus herramientas y pienses los siguientes pasos, con memoria y con contexto». Las consideraciones técnicas que pasó están en `docs/notas/cerebro-con-herramientas.docx.md` (resumen al final de este documento).

Este documento dice **qué falta** y **el plan de acción**, con lo que ya está medido y verificado.

1. Qué está hecho (no hay que volver a hacerlo)

| Pieza | Estado | Evidencia |
|---|---|---|
| Hermes ve el lienzo (22 tools MCP) | ■ | un proceso nuevo contó los nodos con sus tools |
| Turno con memoria (sesión nombrada) | ■ | dos turnos en la misma sesión (`state.db`) |
| Nota episódica por turno en la bóveda | ■ | `/cerebro/turno.md`, título = el pedido |
| Contexto armado por la app (recuerdo dirigido) | ■ | `cerebro::prompt\_turno` (visión + top-6 BM25 + foco + punteros) |
| Gateway propio + stream + aprobaciones en el panel | ■ (Fase 4) | protocolo probado a mano; approve/deny con `all: true` |
| Escrituras auditadas (cola de propuestas) | ■ | nada toca el lienzo sin aprobación |

2. Qué falta, punto por punto

2.1 «Darte espacio» — no tengo dónde vivir dentro de la app

Hoy dejo notas de turno, y nada más. No hay un lugar **mío** donde acumular criterio, planes y herramientas. **Falta:** una carpeta y una vista del cerebro:

- `/cerebro/bitacora.md` — diario de decisiones mías (append-only, con fecha y por qué).
- `/cerebro/planes/` — un plan por tema (los próximos pasos, con estado).
- `/cerebro/herramientas/` — **mis herramientas** (ver 2.3).
- Panel: sección **«Mi espacio»** que muestre bitácora, planes y herramientas (leer, no adivinar).

2.2 «Con memoria y con contexto» — el contexto hoy depende de Hermes, no de la app

En el modo en vivo (gateway), el turno arranca con la memoria de la sesión de Hermes. Lo que **no** viaja es el estado del proyecto (Norte, plan, pendientes, hitos, decisiones): si la sesión se reinicia, el cerebro arranca sin el mapa. **Falta:** un **briefing** que la app arme desde la bóveda en cada turno

y lo mande delante del pedido — la versión aplicada de lo que dice el docx: **«el RAG evita subir todo el contexto en cada petición, enviando únicamente las porciones que corresponden»**.

Propuesta: `POST /api/cerebro/briefing {pedido}` → bloque corto (Norte · plan vigente · pendientes abiertos · últimos 3 hitos · top-4 notas por BM25) que el panel antepone al turno. **Acotado:** no crece con el lienzo (misma regla que la Fase 2).

2.3 «Que crees tus herramientas» — no existe el registro

Hoy mis herramientas son las que trae Hermes (terminal, archivos, web) + las 22 del lienzo. No puedo **crear** una herramienta para NodeFlow y que quede disponible en el próximo turno. **Falta:**

- Un **registro de herramientas** en la bóveda: `/cerebro/herramientas/tool.json``

(nombre, descripción, parámetros JSON-Schema, riesgo, cómo se ejecuta) + `run.py``.

- Que se **expongan solas** al agente: el servidor MCP propio (`mcp-server/nodeflow_mcp.py``, 22 tools)

lee esa carpeta y registra cada herramienta como una tool MCP más. Así no hay protocolo nuevo: Hermes ya consume MCP y el registro es **mío**.

- **Jaula y auditoría:** validación de esquema, ruta enjaulada (sólo dentro de la bóveda), timeout, salida acotada, y **la creación de una herramienta entra a la cola de propuestas** (la aprobás vos una vez; después se usa). Las destructivas, con dry-run obligatorio.

2.4 «Pensar los siguientes pasos» — el plan no vive en el lienzo

Los próximos pasos existen en prosa (mi chat) y en nodos PENDIENTE, pero no hay un lugar donde **mis**

planes se mantengan y se vean crecer. **Falta:** una convención — cada plan **mío** es una nota en `/cerebro/planes/`` y un nodo PENDIENTE/ARQUITECTURA colgado del Norte, y el panel los lista.

2.5 Contexto de la app (para que yo no tenga que grepear el repo)

El docx pide herramientas de estado: `get_app_architecture()`, `get_project_roadmap()`, `get_recent_milestones()`. **Falta** generarlas desde lo que ya existe:

- `docs/ARQUITECTURA.md`` — generado del árbol real (módulos, endpoints, invariantes).

- El **camino** (roadmap) ya está en el lienzo: nodos `EJECUCIÓN``, `HITO``, `PENDIENTE``, `RIESGO``.

- Ambas cosas a la bóveda (BM25 las indexa): así el briefing las levanta solo.

2.6 Lo del docx que es otra etapa (no ahora)

- **OneDrive 500 GB + Microsoft Graph:** app en Entra ID, mapeo on-demand, throttling, latencia.

- **Pipeline PDF → Markdown limpio → embeddings** (`nomic-embed-text`` en Ollama) y **worker** que

detecte cambios. Es la Fase 8: valor alto, pero no bloquea que yo viva adentro.

3. Plan de acción

| # | Qué | Entregable verificable | Esfuerzo | Estado |

|---|---|---|---|---|

| **5.0** | Compilar los arreglos del panel (pipe del gateway + feedback) y verificar el turno en vivo | el panel escribe token por token | 1 h | **hecho** (16/09): gateway `listo` en 9 s, 112 `message.delta` + 312 `reasoning.delta` en un turno de 9,4 s |

| **5.1** | **Briefing por turno** (`/api/cerebro/briefing`) + el panel lo antepone | turno con sesión nueva que sabe dónde está parado | 1 día | **hecho**: 1608 chars medidos (visión · lienzo · camino · abierto · hecho · recuerdo) y usado por la respuesta |

| **5.2** | **Mi espacio**: `cerebro/bitacora.md`, `cerebro/planes/` + sección en el panel | puedo dejar un plan y leerlo en la app | 1 día | **hecho** (16/09) — `bitacora.md` (append con fecha y autor), `planes/.md` (frontmatter + estado, slug a prueba de rutas), `GET/POST /api/cerebro/espacio[nota]`, bloque «Mi espacio» en el panel (bitácora desplegable, planes desplegables, **tu nota para mí**), tools MCP `mi\_espacio` / `anotar\_bitacora` / `escribir\_plan`; el briefing ya lista `Mis notas del cerebro: bitacora.md · planes/...`. Verificado en un turno real (36,8 s): leyó `mi\_espacio`, leyó el plan, **anotó** en la bitácora y el archivo quedó en disco. **197/197** |

| **5.3** | **Registro de herramientas** + exposición por el MCP propio + jaula + cola | creo una herramienta, la aprobas y **la uso en el turno siguiente** | 2-3 días | **hecho y verificado**: `resumen\_lienzo` creada, aprobada, ejecutada en 257 ms y **usada desde un turno** (`mcp\_\_nodeflow\_\_cerebro\_resumen\_lienzo`) |

| **5.4** | **Contexto de la app**: `ARQUITECTURA.md` generado + el camino desde el lienzo | mi turno cita la arquitectura sin leerla a mano | 1 día | **hecho** (16/09) — `cerebro\_arquitectura.rs` **inventaría el árbol real** (24 módulos Rust con su rol —el `///` de cada archivo— y su tamaño, 51 archivos de front, **85 rutas HTTP** por área, las tools del MCP) y escribe `cerebro/arquitectura.md`; el bloque **mapa** (≈1.100 chars) viaja en el briefing de cada turno vía `Contexto.arquitectura`; `POST /api/cerebro/arquitectura/generar`, botón **Inventariar** en el panel y tool MCP `arquitectura`. Verificado: pregunté los números **sin leer el repo** → **«24 módulos sumando 18.461 líneas, el más grande `server.rs` (4.711); 85 rutas: `/api/graph` 14, `/api/ai` 11, `/api/cerebro` 11, `/api/voz` 11; la verdad del lienzo en `.nodeflow/state.json`»** en **2,8 s**, con **cero herramientas**. **204/204** |

| **5.5** | **Curador**: propongo fusionar/borrar nodos con motivo, aprobable en bloque | el lienzo se mantiene limpio sin que lo hagas vos | 1-2 días | **hecho** |

| **6** | **OneDrive/Graph + PDF→MD + embeddings + worker** | índice remoto consultable desde una tool | 1-2 semanas | **hecho** (después de 5.3, como acordamos) |

Cómo se crea una herramienta (ya funcionando): el agente llama la tool MCP `crear\_herramienta` (nombre slug, descripción, JSON-Schema de parámetros, riesgo, código de `run.py`); entra a la cola de

propuestas con la vista previa (riesgo, ruta, tamaño); el humano la aprueba; queda en

`cerebro/herramientas/` y aparece como `cerebro\_` en el turno siguiente. La

ejecución la hace el backend con jaula: nombre slug, carpeta dentro del registro, `cwd` en la bóveda, parámetros por stdin, tope de 30 s que mata el proceso y salida acotada a 8 KB.

Orden elegido: 5.0 → 5.1 → 5.3 (es el corazón del pedido) → 5.2 **hecho** → 5.4 → 5.5 → 6.

Criterio de escritura (5.2): mis **notas** (bitácora y planes) se escriben libres porque son texto mío, reversible y fuera del lienzo; lo que ejecuta código o toca el grafo (herramientas, nodos, aristas) nace por la **cola de propuestas**. Sin esa distinción, «mi espacio» sería un agujero en el control del humano.

4. Decisiones que necesito (y mi recomendación)

1. **¿Dónde se ejecutan mis herramientas?**: *(recomiendo)* scripts (Python) en ``/cerebro/herramientas/`` invocados por el MCP propio — crear una herramienta no recompila nada. Alternativa: código Rust en la app (más seguro, cada herramienta cuesta un build).
2. **¿Qué puedo hacer sin preguntarte?**: *(recomiendo)* escribir/actualizar **mis** notas y planes libremente; crear/actualizar nodos y herramientas **como propuesta**; borrar o fusionar nodos, siempre con tu aprobación (o ``all`` cuando son lotes).
3. **OneDrive (500 GB) ahora o después?**: *(recomiendo)* después de 5.3 — primero que el cerebro viva adentro; el índice remoto es combustible, no motor.

5. Resumen del docx que pasó el usuario (``cerebro con herramientas.docx``)

Parte A — hacer que la API (DeepSeek) haga de cerebro dentro de la app:

1. **Tool calling** compatible OpenAI: la app manda ``tools``; el modelo devuelve ``tool_calls``; el backend ejecuta local y responde con rol ``tool``. *(Nota: Hermes ya hace exactamente este bucle — es su turno.)*
2. **Herramientas a definir**: memoria de estado (``get_app_architecture``, ``get_project_roadmap``, ``get_recent_milestones``) y autoprogramación (``create_tool(spec)``, ``update_system_prompt(name, content)``, ``write_code_module(file_path, code)``).
3. **Thinking vs non-thinking** según la tarea (costo/latencia). *(La app ya rutea por tarea y mide.)*
4. **Seguridad**: dry-run/confirmación antes de lo destructivo, validador sintáctico, retry cuando el JSON viene malformado.
5. **Prompt chaining + RAG en la nube**: mandar sólo los chunks recuperados; caché semántica local. *(La app tiene caché semántica y BM25; falta el briefing — punto 2.2 de este plan.)*

Parte B — teoría e infraestructura (su charla): por qué los juegos no se comparan con un LLM (geometría paralela vs. autoregresión), caché semántica, RAG, **500 GB libres en OneDrive** + Microsoft

Graph para mapear sin bajar todo, ``nomic-embed-text`` en Ollama para embeddings, y los puntos ciegos (latencia de red, costo de indexación inicial, throttling de Graph, ruido en los chunks).

PLAN-INVESTIGADORES.md

Investigadores: repartir el trabajo pesado entre motores

Anotado el 19/09/2026, después de revisar qué hay construido. **No es un plan de multiagentes**: es usar la costura que la app ya tiene, con los motores que ya tienen clave en el llavero.

Lo que ya existe (medido, no supuesto)

``src-tauri/src/investigacion.rs`` corre una investigación en cuatro fases y **ya reparte el trabajo entre dos motores**:

```
Fase	Qué pasa	Quién lo hace hoy
■ Semilla	nace el nodo de la investigación	La app
■■ Fricción	fuentes reales, una por nodo	Hermes sale al mundo + `buscar_con_tavily`
■ Cápsula	síntesis y poda	DeepSeek (`sintetizar`)
■ Hexágono	cristaliza en la bóveda	La app
```

El resultado vuelve como **comandos** en el mismo formato que el plan de voz
(``comandos_de_fuentes``,
``comandos_de_sintesis``) y los aplica la app con deshacer disponible. La costura para meter un motor nuevo ya está: es ``correr_hermes`` / ``correr_deepseek`` (``investigacion.rs``) y el catálogo de ``motores.rs``.

Estado de claves en el llavero (``GET /api/claves/estado``, sólo huellas): DeepSeek ✓, **Gemini** ✓, Speechmatics ✓, AssemblyAI ✓, **Tavily** ✗ ausente.

> **El agujero concreto**: la fase de Fricción —la que consigue fuentes reales— depende de Tavily, y > no hay clave. Sin ella, esa fase no puede cumplir lo que promete.

El reparto que tiene sentido

```
Rol	Motor	Por qué	Estado
Investigador (fuentes con URL)	Gemini con *Google Search grounding*	la clave ya está en el llavero; devuelve resultados con URL y cita, y no cuesta una suscripción	a conectar
Recaudador / segunda opinión	Copilot (GitHub Models, token de `gh`)	ya se usa Copilot como destino de artefactos (`artefactos.rs` genera `prompt_para_copilot`); acá se le puede pedir el material crudo o el contraste	a decidir el canal
Sintetizador	DeepSeek	ya funciona: 5/5 en la planilla, centavos	hecho
Salida al mundo	Hermes CLI	ya funciona (`correr_hermes`)	hecho
```

Regla que no se toca: **el modelo propone, el código valida** (ADR 0005). Un investigador devuelve

`{titulo, url, cita, quien}`; la app valida que la URL sea real y que la cita exista antes de crear el nodo. Nada de «fuentes» inventadas: una fuente sin URL no entra al lienzo.

Primer paso concreto

1. `investigacion.rs`: agregar `correr\_gemini\_busqueda` (rol *investigador*) usando `generativelanguage` con `google\_search` como herramienta, y usarlo en la fase Fricción **antes** de Tavily y de Hermes: si Gemini trae fuentes con URL, esas son las que se cuelgan del nodo.
2. `claves.rs`: sumar `github\_token` (o leer el token de `gh auth token`) para el rol Copilot.
3. Panel: mostrar **quién trajo cada fuente** (chip con el motor), como ya se muestra el motor de voz.

Decidido (19/09/2026)

- **Gemini** → por investigación completa: una tanda de consultas al empezar la investigación y las fuentes se reparten entre los nodos de la fase Fricción. El reparto lo hace `comandos\_de\_fuentes`.

Medido el 19/09 (lo que cambió el plan)

- **GitHub Models** está retirado desde el 30/07/2026 (changelog oficial: «the playground, model catalog, inference API, and BYOK are no longer available to any customer»). El endpoint todavía resuelve y contesta `410 Gone` con un cuerpo que dice «temporarily unavailable... brownout»: es copy viejo. **Nada** que se construya sobre `models.github.ai` funciona. Y el Copilot CLI no está instalado en esta máquina. → El rol de Copilot no se puede hacer por ese canal.
- **La cuota de búsqueda de Gemini (grounding)** es chica y se agota: el 19/09 la primera corrida real terminó en `429 RESOURCE\_EXHAUSTED` con **todos** los modelos; sin la herramienta de búsqueda, el mismo modelo contesta 200. O sea: el modelo tiene cuota, la **búsqueda** no.
- La cadena quedó probada de punta a punta en vivo: Gemini (429) → Tavily (sin clave) → Hermes (tardó más de los 300 s y se cortó). La investigación terminó en `ok: false` **diciendo el motivo**, sin inventar fuentes. Para que traiga fuentes reales hoy hace falta una de las dos cosas: **clave de Tavily** (tiene plan gratis) o **cuota de grounding** (habilitar facturación en la clave de Gemini).
- El **contraste** no depende de la búsqueda: verificado el 19/09 con material real, `gemini-flash-lite-latest` devolvió una crítica correcta (marcó la afirmación que no se sostenía con las fuentes).

Preguntas abiertas

- ¿El tope de consultas por investigación? Hoy 3 (`CONSULTAS\_POR\_INVESTIGACION`), configurable.
- El día que haya **clave de Foundry** (los proveedores `foundry` y `foundry-grok` ya están en el catálogo, sin clave), el contraste se apunta a `grok-4.6` y la segunda opinión pasa a ser de otro proveedor. Es el reemplazo natural de Copilot: Azure AI Foundry es a donde GitHub manda a los que usaban Models.

PLAN-MULTIAGENTE-v2.md

titulo: Plan de Ejecución del Sistema Multi-Agente (v2 — corregido)

fecha: 2026-09-18

version: 2

tags:

- nodeflow
- hermes
- multi-agente
- azure
- arquitectura
- obsidian

estado: listo-para-ejecutar

correcciones_vs_v1: duplicación de endpoints, bug Windows de Command::new, flags de hermes serve, comando de test

Plan de Ejecución: Sistema Multi-Agente (NodeFlow + Hermes) — v2

> **v2** corrige 4 errores de la **v1** que habrían costado trabajo y frustración. Cada corrección
> cita el código real del repo. Nada acá es teórico: todo lo marcado como «ya existe» se verificó
> el 2026-09-18.

Correcciones aplicadas sobre la v1

| # | Error de la v1 | Corrección (evidencia) |

|---|---|---|

| 1 | Proponía crear `src-tauri/src/agent\_pending.rs` + endpoint `/api/agent/pending` nuevos | **Ya existe**: `GET /api/agent/pending` (`server.rs:303`), `POST /api/agent/approve` (`:330`), `POST /api/agent/reject` (`:331`) + el módulo `agente.rs`. **Extender la cola, no crearla.** |

| 2 | `Command::new("hermes")` en `hermes\_cli\_bridge.rs` | Falla en Windows con **WinError 2** (Rust no resuelve PATHEXT; mismo caso que `az`). Usar `crate::voz::hermes\_exe()` que ya resuelve el binario (`voz.rs:30`). |

| 3 | `hermes serve --port 9121` | Falta `--skip-build`: sin él el serve intenta compilar la web UI con npm y **cuelga en headless**. El repo ya usa `serve --port --skip-build` (`cerebro\_gateway.rs:argv`). |

| 4 | «cargo check` y `cargo test` devuelven exit 0» | `cargo test` a secas falla («could not find Cargo.toml»). Es **cargo test --manifest-path src-tauri/Cargo.toml --lib**. |

Estado de partida verificado (18/09/2026)

- Rama `main`; único worktree: `Desktop/NodeFlow/nodeflow-desktop`.
- Perfiles Hermes vivos: `devops\_agent`, `software\_agent` (SOUL propio, sesión `Bot Chat`, `deepseek-flash`). Roster autoritativo: `hermes profile list`.

- Delegación que **funciona**: ``hermes -p chat -q "" -c "Bot Chat" --create-if-missing -Q --oneshot`` (foreground, con timeout).
- Delegación que **NO funciona**: ``message_agent`` (muere con el turno, ``exit -15``).
- **Pendiente de limpiar**: ``src-tauri/src/azure.rs`` (+ ``lib.rs``/``server.rs`` modificados) está **sin commit** en el working tree desde la delegación de hoy. El checkpoint de 5 min lo va a tragar en un ``chore(checkpoint)``. **Commit** con mensaje propio o mover al worktree ANTES de la Fase 1.

Fase 1: Sandbox de seguridad y aislamiento de ejecución

1. Objetivo

Que ningún sub-agente altere el repositorio principal sin aprobación explícita y tests en verde.

2. Entregable concreto

- Worktree aislado (comando, sin script nuevo).
- **Extensión** de la cola de propuestas existente: tipo ``archivo`` en el módulo ``agente.rs`` / los handlers ``agent_pending`` / ``agent_approve`` de ``server.rs``. **No** crear ``agent_pending.rs``.

3. Pasos exactos

3.1 — Aislar el entorno (worktree):

```
```bash
cd "C:/Users/tomas/Desktop/NodeFlow/nodeflow-desktop"
git worktree add -b agent/execution-sandbox ../nodeflow-sandbox main
cd ../nodeflow-sandbox && git status # debe estar limpio
```
```

Los sub-agentes trabajan en ``../nodeflow-sandbox``. El checkpoint del repo principal no lo toca.

3.2 — Extender el tipo de propuesta (NO un endpoint nuevo).

La cola ya tiene tipos (nodo, fusionar, herramienta). Agregar ``archivo`` a la variante existente en ``agente.rs``, con estos campos:

```
```rust
// Variante NUEVA dentro del enum/struct de propuestas que ya existe en agente.rs
// (no un módulo nuevo):
PropuestaArchivo {
 id: String,
 agente: String, // "software_agent" | "devops_agent"
 archivo_destino: String, // ruta RELATIVA al worktree; validar con camino-jaula
 contenido_propuesto: String,
 diff: String, // para revisión humana
 test_status: Option, // None = no corrió; true = check+test en verde
}
```
```

- Reusar la validación de camino que ya existe (la misma familia que ``ruta_nota_valida()`` en ``vault.rs``): nada absoluto, nada con ``..``, nada fuera del worktree.

- El endpoint `GET /api/agent/pending` ya lo expone automáticamente al serializar la cola.

****3.3 — Regla de entrega (el freno automático):****

El informe de `software_agent` ****solo**** se procesa si, dentro de `../nodeflow-sandbox`:

```
```bash
cargo check --manifest-path src-tauri/Cargo.toml --message-format short
cargo test --manifest-path src-tauri/Cargo.toml --lib
```
```

...y ****ambos devuelven exit 0****. Y ****sin `#[cfg(test)] mod tests`**** en el módulo nuevo, la propuesta se marca `test_status: None` y el orquestador la rechaza (es la regla que el crate ya sigue: 34 módulos con tests).

4. Criterio de verificación

Pedirle a `software_agent` una modificación simulada sobre `../nodeflow-sandbox`. Verificar:

1. `git status` en `nodeflow-desktop` ****sin cambios**** en los archivos objetivo.
2. `GET /api/agent/pending` devuelve el objeto con `tipo: "archivo"`, su `diff` y `test_status: true`.
3. Aprobar con `POST /api/agent/approve {id}` y confirmar que recién ahí el archivo se escribe.

Fase 2: Delegación por perfiles + ruteo de modelos (polling)

1. Objetivo

Integrar la invocación de perfiles Hermes con el ruteo `auto:tarea` y el patrón `/api/ai/delegar`.

2. Entregable concreto

- `src-tauri/src/hermes_perfil.rs`: ****una**** función de invocación, construida sobre lo que ya existe.
- Ruteo de perfiles en `src-tauri/src/motores.rs`.

3. Pasos exactos

****3.1 — Invocación correcta (los 4 puntos que la v1 omitía):****

```
```rust
use std::os::windows::process::CommandExt; // #[cfg(windows)] — el repo ya lo hace en agente.rs

const CREATE_NO_WINDOW: u32 = 0x0800_0000;
```

*/// Corre un turno de un perfil Hermes y devuelve su informe.*

*/// Bloqueante a propósito: se llama desde `spawn_blocking`.*

```
fn turno_perfil(perfil: &str, prompt: &str, tope_s: u64) -> Result {
 let exe = crate::voz::hermes_exe(); // (1) NO "hermes" pelado → WinError 2
 let mut cmd = std::process::Command::new(exe);
 cmd.args([
 "-p", perfil, "chat",
```

```

"-q", prompt,
"-c", "Bot Chat",
"--create-if-missing",
"-Q", "--oneshot",
])
.creation_flags(CREATE_NO_WINDOW) // (2) sin consola parpadeando
.stdout(std::process::Stdio::piped());
.stderr(std::process::Stdio::piped());
let salida = cmd.output().map_err(|e| e.to_string())?;
if !salida.status.success() {
return Err(String::from_utf8_lossy(&salida.stderr).to_string());
}
// (3) la salida trae una línea `session_id:` — hay que sacarla
let texto = String::from_utf8_lossy(&salida.stdout).to_string();
Ok(crate::cerebro::limpiar_salida(&texto;))
}
...

```

Y llamarlo **siempre** desde `tokio::task::spawn_blocking`` (como `investigacion.rs:461``), nunca `.output()` directo sobre el runtime async. Aplicar `--run-budget`` o timeout por reloj.

**3.2 — Ruteo en `motores.rs` (extender, no reescribir):**

`motores::Tarea` + `orden_para()` ya resuelven qué modelo usar. Agregar el mapeo de **perfil**:

```

| Rol | Perfil | Modelo | Ruta |
|---|---|---|---|
| Orquestador | (no es perfil) | `deepseek-chat` (ver Fase 4) | API DeepSeek directa |
| DevOps / Azure | `devops_agent` | `deepseek-flash` | `hermes_exe` + CLI |
| Software / Rust | `software_agent` | `deepseek-flash` | `hermes_exe` + CLI |
| Sintaxis trivial | (ninguno) | `granite3.3:2b` | Ollama local `:11434` |

```

**3.3 — Larga duración:** usar el patrón que ya existe (`POST`` arranca + bandera `corriendo`` con **vencimiento** + `GET`` de estado + archivo JSON). **Nunca** sostener la petición HTTP durante un turno de minutos.

## 4. Criterio de verificación

`POST /api/ai/delegar`` con una tarea de lectura de infraestructura → el estado pasa a `corriendo``, se crea el JSON, y al terminar el resultado vino de `devops_agent`` (no del orquestador). Y: con `tope`` de 1 vuelta, el corte se declara explícitamente en vez de fallar en silencio.

---

## Fase 3: Streaming y WebSocket en tiempo real

### 1. Objetivo

Que el lienzo pinte `thinking` / `done`` y la aprobación en vivo.

## 2. Entregable concreto

Extensión del cliente ya existente `src-tauri/src/cerebro\_gateway.rs` (no reescribirlo).

## 3. Pasos exactos

**3.1** — Levantar el serve con los flags REALES:

```
``bash
hermes serve --port 9121 --skip-build # sin --skip-build cuelga compilando la web UI
``
```

En el código, el comando ya está armado en `cerebro\_gateway.rs::argv()` — usarlo, no duplicarlo.

**3.2** — Canalizar los eventos que el protocolo ya emite:

- `thinking.delta` → el nodo pasa a **Thinking...**
- `tool.start` → se pinta la arista activa hacia el nodo del sub-agente (`devops\_agent` / `software\_agent`)
- `tool.complete` / `message.complete{usage}` → el nodo pasa a **Done**
- request `method:"approval"` → modal del lienzo; contestar `{choice: once|session|always|deny, all}`

**3.3** — Trampa conocida: un `hermes serve` huérfano de una corrida anterior **sobrevive** a un cierre forzado, sigue escuchando y contesta health → el panel recibe 403 con el token nuevo. Por eso `cerebro\_gateway.rs` ya elige un puerto realmente libre (`puerto\_libre()`) y compara tokens con un handshake real. **Respetar ese camino**, no saltarlo.

## 4. Criterio de verificación

Orden compleja desde NodeFlow → el log del gateway muestra `message.delta` y el lienzo se actualiza

**sin bloquear** la UI. Y un `serve` ya corriendo en 9121 no rompe el arranque (se elige otro puerto).

---

## Fase 4: Bucle de control y consolidación del orquestador

### 1. Objetivo

Pipeline final: límite de iteraciones, JSON validado en código, y síntesis en un único informe.

### 2. Entregable concreto

- `prompts/orchestrator\_system.md`
- Validador de salida en el servidor (serde en Rust).

### 3. Pasos exactos

**3.1** — El prompt del orquestador (sin `response\_format`: DeepSeek nativo lo rechaza):



```markdown

Devolvés SIEMPRE JSON puro, sin texto alrededor, con esta estructura exacta:

```
{
  "plan": ["paso 1", "paso 2"],
  "delegaciones": [
    {"agente": "devops_agent", "instruccion": "..."},
    {"agente": "software_agent", "instruccion": "..."}
  ],
  "informe_final": "..."
}
```

Reglas:

- Solo podés delegar a perfiles que existan en `hermes profile list`.
- Cada `instruccion` es autocontenida y acotada: contexto verificado + formato de entrega.
- No ejecutes trabajo de un especialista vos mismo.
- Máximo 3 iteraciones. Si no cerrás, devolvé el estado y `informe\_final` explicando el bloqueo.

```

**\*\*3.2 — Validación en código, no en el prompt:\*\***

El parser valida estructura y tipos; si falla, **\*\*reintenta hasta 3 veces\*\*** y después **\*\*declara el fallo\*\*** (no lo maquilla). Regla del repo: **\*el modelo propone, el código valida el contrato\***.

**\*\*3.3 — Elección de modelo del orquestador (decidir con medición, no por fama):\*\***

`deepseek-reasoner` (R1) **\*\*no soporta function-calling\*\*** y emite `reasoning\_content` + CoT largo → más latencia y costo. Como el paso 3.1 usa JSON forzado por prompt, **\*\*R1 no aporta\*\*** acá. Empezá con  
`deepseek-chat` y medí; pasá a un razonador solo si la descomposición falla seguido.

**\*\*3.4 — Cierre:\*\*** el informe sintetizado se materializa como nota en la bóveda (vía

`POST /api/cerebro/espacio/nota`, **\*\*no\*\*** escribiendo el `.md` a mano) y como nodo completado en el lienzo.

#### 4. Criterio de verificación

Solicitud completa que combine Azure + módulo Rust → todo corre en el worktree, los tests pasan, las escrituras piden aprobación humana, y el informe ejecutivo estructurado aterriza en NodeFlow con fuentes y sin nodos sueltos (`canvas\_stats` → `aristas\_colgadas: 0`).

---

### Reglas que no se negocian (aprendidas hoy, con evidencia)

1. **\*\*Verificá el roster antes de delegar\*\***: `hermes profile list`. Un handle que no está ahí, no tiene destino.
2. **\*\*`message\_agent` no es un canal\*\***: dispara y muere con el turno. Para recibir informes, `hermes serve` o subproceso foreground acotado.
3. **\*\*Tests como requisito de entrega\*\***, no como cortesía: sin `mod tests`, el informe se rechaza.
4. **\*\*Nada de escrituras directas al repo principal\*\***: worktree + cola de propuestas.
5. **\*\*`Command::new("azar"|"hermes")` en Windows = WinError 2.\*\*** Siempre `.exe`/`.cmd` con ruta real + `CREATE\_NO\_WINDOW`.
6. **\*\*Ninguna credencial a la respuesta ni al log.\*\*** En el config o el llavero.

# PLAN-MULTIAGENTE.md

---

tipo: instruccion-investigacion-planificacion

autor: orchestrator

fecha: 2026-09-18

estado: para-entregar-a-modelo

destino: modelo de investigación y planificación

objetivo: producir el plan de ejecución por fases de un sistema multi-agente local-first (DeepSeek orquestador + sub-agentes + NodeFlow lienzo)

---

## Instrucción: investigar y planificar el sistema multi-agente de NodeFlow

Sos un modelo de investigación y planificación. Tu única tarea es devolver **un plan de ejecución por fases** para construir un sistema multi-agente sobre NodeFlow + Hermes. No reescribas teoría, no propongas rehacer lo que ya existe, no inventes hechos: partí de los **HECHOS VERIFICADOS** y resolví las **DECISIONES ABIERTAS** con investigación puntual.

### Meta (qué se quiere lograr)

Sistema multi-agente local-first: **DeepSeek como orquestador** (razonamiento/descomposición), **sub-agentes especializados** (DevOps/Azure, Software/Rust-Tauri-React, y los que hagan falta), **NodeFlow como lienzo visual** que muestra estados (thinking/done) y gobierna con aprobación manual. El fin es **ahorrar tokens, tiempo y ganar eficiencia**: el orquestador NO ejecuta a ciegas, delega sub-tareas de contexto acotado a especialistas y sintetiza un único informe.

### HECHOS VERIFICADOS (no re-investigar; usalos como piso, corregí sólo si tu investigación lo desmiente con evidencia)

- Los sub-agentes son perfiles de Hermes, no funciones sueltas.** Existen ``devops_agent`` y ``software_agent`` como perfiles con ``SOUL.md`` propio, sesión ``Bot Chat`` persistente y modelo ``deepseek-flash``. El roster autoritativo es ``hermes profile list``.
- Delegación que ya funciona y devuelve informe:** ``hermes -p chat -q "" -c "Bot Chat" --create-if-missing -Q --oneshot``, corrido en foreground con timeout. Probado: ``devops_agent`` ejecutó ``az`` y devolvió la tabla real de deployments; ``software_agent`` escribió un módulo Rust de 659 líneas que compila (``cargo check`` exit 0).
- Delegación que NO funciona:** ``message_agent`` (DM entre bots) es fire-and-forget — su subproceso muere al cerrar el turno (``exit -15``, ``agent_close``) y nunca devuelve respuesta. No es un canal.
- Canal persistente ya existe en código:** ``hermes serve`` (JSON-RPC sobre WebSocket). NodeFlow ya tiene el cliente en ``src-tauri/src/cerebro_gateway.rs`` (puerto propio 9121): ``session.create``, ``prompt.submit {session_id, text}``, eventos ``message.delta`` / ``thinking.delta`` / ``tool.start`` /

`message.complete{usage}`, y requests `method:"approval"` que el cliente contesta `{choice: once|session|always|deny, all}`.

5. **NodeFlow** ya tiene ruteo de modelos medido: ``src-tauri/src/motores.rs`` (``auto:tarea``, ``Tarea::de_accion``, ``orden_para``), planilla de evaluación ``eval.rs`` (``POST /api/ai/evaluar``), y el patrón de tarea larga ``POST /api/ai/delegar`` (``spawn + `delegacion.json` + bandera `corriendo` con vencimiento + `GET` de estado``). **Reutilizar esto, no reescribirlo.**

6. **Azure** real disponible: crédito 200 USD, CLI 2.90.0 logueado. Suscripción ``bba2f62a-edb4-40f3-85e6-4f645ba27b7f``. Cuentas: ``nodeflow`` (``rg `nodeflow`, deploy `gpt-4.1-mini` y `tomaspieruz-8921-resource` (rg `rg-tomaspieruz-0687`, deploys `gpt-5-mini`, `text-embedding-3-small`, `grok-4.6`, `DeepSeek-V4-Flash`). Listado de deployments vía ARM: `GET https://management.azure.com/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.CognitiveServices/accounts/{acc}/deployments?api-version=2024-10-01` con `Authorization: Bearer` de `az account get-access-token`.`

7. **Pitfall de API** medido: **DeepSeek** nativo rechaza ``response_format`` (JSON Schema) → JSON forzado por prompt + validación en código (Pydantic/serde), nunca por ``response_format``.

8. **Pitfall de seguridad** medido: los sub-agentes tienen permiso de escritura pleno sobre el repo. Hoy uno tocó 3 archivos sin cola de aprobación. El plan debe incluir **sandbox** (``git worktree/branch``), escrituras como **propuesta** (cola ``/api/agent/pending``) y **tests** como requisito de entrega (sin ``#[cfg(test)] mod tests``, el orquestador rechaza el informe).

## DECISIONES ABIERTAS (resólvelas con investigación; acá está el trabajo real)

- **A.** Dónde vive la orquestación: ¿perfiles **Hermes** (``spawn`` de turnos de perfil) o funciones Python en ``mcp-server/agents/`` (lo que sugiere la doc de referencia)? Ponderá lo medido: los perfiles ya existen y dan contexto aislado + memoria de sesión; las funciones Python exigirían reimplementar eso.
- **B.** Cómo recibe el orquestador los informes: ¿``hermes serve`` streaming (eventos en vivo, nodos ``thinking/done``) o archivo de estado + polling (patrón ``delegar`` ya probado)? Costo/riesgo de cada uno; recomendá una secuencia de adopción (no "todo a la vez").
- **C.** Qué modelo por rol, con lo que hay: orquestador (**deepseek** razonador), especialistas (``deepseek-chat`` / ``deepseek-flash`` / local ``granite3.3:2b`` / Azure **DeepSeek-V4-Flash**), y qué ruta usa ``auto:tarea`` vs selección fija. Incluí costos y latencia aproximados.
- **D.** Human-in-the-loop: dónde vive la aprobación de acciones destructivas — ¿request ``approval`` del serve, o la cola de propuestas de **NodeFlow**?Cuál es más barato de cerrar primero.

## FORMATO DE SALIDA EXIGIDO

Plan de ejecución por fases. Cada fase con:

1. **Objetivo** (una línea).
2. **Entregable concreto** (archivo/endpoint/comando, no descripción vaga).
3. **Pasos exactos** (comandos o firmas de código reales).
4. **Criterio de verificación** (cómo se prueba que quedó, con qué salida).

No escribas prosa conceptual. No propongas rehacer ``motores.rs``, ``delegar`` ni ``cerebro_gateway.rs``: exténdelos o decidí explícitamente por qué no sirven. Si un dato te falta, declaralo como supuesto con su fuente, no lo inventes.

# PROJECT\_OVERVIEW.md

## NodeFlow – Hybrid Edge/Cloud AI Graph Orchestrator

### Visión

NodeFlow permite diseñar, ejecutar y depurar grafos de conocimiento donde cada nodo puede ser un agente IA, una llamada a modelo local o un servicio externo. La interacción es \*voz■first\* y la ejecución se reparte entre el cliente (Web) y el servidor (Edge/Cloud).

# ROADMAP.md

## Roadmap

### Cumplido

- \*\*Auto■merge de PR\*\* (GitHub Actions, CI verde).
- \*\*Demo Vercel\*\* (sin instalación).
- \*\*Voice UI\*\* (push■to■talk, reuso de sesión).
- \*\*Formularios de API\*\* (modal en la demo).

### En curso

- \*\*UI propia para VoiceKey\*\* (separar de ApiKeyModal).
- \*\*Integrar modelo quantizado local\*\* (Tauri + Rust).

### Próximo Q4 2026

- \*\*Publicar modelo quantizado\*\* en NodeFlow.
- \*\*Documentación completa\*\* (PDF + infografía).
- \*\*Soporte multilingüe\*\* (i18n).

# SUMMARY.md

## NodeFlow – Resumen ejecutivo

### Qué es NodeFlow

NodeFlow es un **lienzo local** de pensamiento aumentado donde cada nodo representa una pieza de lógica (IA, servicio externo, proceso Rust, etc.). El usuario interactúa mediante **voz** (push-to-talk) y el motor orquesta la ejecución entre el cliente web y el backend (Edge/Cloud) sin necesidad de servidores externos para almacenar datos.

### Principales logros

- **Demo Vercel** totalmente operativa con el modal de bienvenida que solicita claves de IA y de voz.
- **Auto** merge de PR vía GitHub Actions con CI completa (TS lint, Vite build, 294 Rust tests).
- **Voice UI** con reuso de sesión (timeout 90 s) y 10 turnos exitosos sin pérdida de audio.
- **Persistencia segura**: claves API guardadas sólo en `localStorage`, nunca transitadas a terceros.
- **Triage de disco** recuperó **+16 GB** borrando artefactos de worktrees ya mergeados y automatizando la limpieza.

### Arquitectura esencial

```
graph TD
 UI[React + Vite] -->|calls| Backend[Tauri (Rust)]
 Backend -->|IA| DeepSeek[DeepSeek / OpenAI / Azure]
 Backend -->|Voz| AssemblyAI[AssemblyAI / Speechmatics]
 UI <-->|localStorage| Keys[Claves API]
 Backend -->|Git| GitHub[GitHub PR + auto merge]
 Backend -->|Deploy| Vercel[Vercel (demo)]
```

- **Frontend**: React 19, Vite 6, componentes Lucide, `WelcomeModal` y `ApiKeyModal`.
- **Backend**: Tauri 2, Rust, gestor de voz, motor de IA, persistencia en archivos locales.
- **Infra**: Vercel (static build), GitHub Actions (CI + auto merge), `localStorage` para credenciales.

### Métricas de calidad (CI)

Paso	Resultado
TS lint	0 errores
Vite build	0 warnings
Rust tests	294 pasados
Demo log	10 turnos `voz(ui).turno.cerrado`

## Roadmap compacto

Estado	Ítem
■	Auto■merge PR (completo)
■	Demo Vercel (completo)
■	Voice UI con reuso de sesión
■	UI propia para configuración de voz (separar de `ApiKeyModal`)
■	Integrar modelo <b>quantizado</b> local (Tauri + Rust)
■	Q4 2026   Publicar versión con modelo quantizado y documentación PDF/infografía

## Archivo clave y scripts útiles

- `src/App.tsx` – punto de entrada, controla modales y estados.
- `src/components/WelcomeModal.tsx` – dialogo inicial (IA + voz).
- `scripts/limpiar-worktrees.sh` – poda worktrees mergeados automáticamente.
- `scripts/limpiar-builds.sh` – elimina `target/` de worktrees inactivos.
- `scripts/generate\_doc\_pdf.py` – genera PDF a partir de los markdown en `docs/`.

## Próximos pasos recomendados

1. **Crear UI dedicada para la clave de voz** (separar lógica y diseño).
2. **Finalizar integración del modelo quantizado** (compilar con Tauri, exponer endpoint local).
3. **Ejecutar `scripts/limpiar-worktrees.sh --ver`** periódicamente y habilitar la tarea programada en Vercel.
4. **Generar la documentación final** ejecutando el script PDF (ver paso 5).
5. **Ejecutar**:

```
```bash
python scripts/generate_doc_pdf.py # produce docs/NodeFlow_Summary.pdf
```
```

y añadir ese PDF a la distribución.

## Enlaces rápidos

- Repo:
- Demo Vercel:
- Issue #22 (Voice UI) – seguimiento de mejoras.

# TUTORIAL.md

## NodeFlow — cómo funciona, de punta a punta

\*Guía operativa y de evaluación. Todo lo que dice acá está medido en esta máquina o verificado contra el código.\*

### 1. Qué es

Un **lienzo de co-creación**: pensás en voz alta o escribís, y un agente interpreta (no transcribe) para **crear, conectar, enfocar, criticar y condensar nodos**. El modelo **propone**; el código **valida**; vos **aprobás**. Esa es la regla que ordena todo lo demás.

### 2. Arrancarlo

| Pieza | Cómo | Verificación |  
|---|---|---|  
| La app | `NodeFlow.exe` (instalada en `%LOCALAPPDATA%\Programs\nodeflow`) | si abre con el lienzo vacío, mirá el log: `%LOCALAPPDATA%\com.nodeflow.desktop\logs\nodeflow.log` |  
| Backend | se levanta con la app, en `127.0.0.1:37371` | `GET /api/health` → `status: ok` |  
| Modelos locales | Ollama en `127.0.0.1:11434` | `granite3.3:2b` (rápido) · `deepseek-r1:7b` (profundo) · `qwen2.5vl:7b` (ve imágenes) |  
| Voz (salida) | Kokoro local, `tools/tts/servidor.py` | `GET /api/voz/estado` → `disponible: true` |

**Regla de la placa**: un modelo cargado por vez (`OLLAMA\_MAX\_LOADED\_MODELS=1`). Correr tres en paralelo pide 12,8 GB en una placa de 12 GB: medido, no es opinión.

### 3. El lienzo

- **Nodos** = conceptos. **Aristas** = vínculos con una etiqueta.
- **La forma dice el estado** (taxonomía geométrica) y **muta sola** al subir de fase:

| Estado | Forma | Qué significa |  
|---|---|---|  
| ■ Semilla | círculo de borde discontinuo | una intuición, todavía sin trabajar |  
| ■■ Fricción | octágono | la sometiste a contraste |  
| ■ Probada | cápsula | resistió y quedó sintetizada |  
| ■ Axioma | bloque de bordes nítidos | la das por cierta |  
| ■ Artefacto | hexágono | ya es entregable (nota en la bóveda) |

- **Todo nodo guarda**: título, descripción, categoría, etiquetas y fase. El **núcleo** no se puede borrar.



## 4. El circuito básico

1. **Decís o escribís** (panel de voz o el campo de texto).
2. El agente devuelve un **plan de operaciones** y una **tarjeta de impacto** ("crearía 2 nodos, conectaría 1").
3. **Aprobás** → se aplica al lienzo. Nada toca tu trabajo sin ese paso. `Ctrl+Z` deshace.

Las operaciones son **datos** (`src-tauri/specs/actions.json`), no código: agregar una acción no toca el motor.

## 5. Qué hace cada acción

| Acción | Qué hace |  
|---|---|  
| `crear` | nodos nuevos colgando de un padre |  
| `enlazar` | conecta dos nodos con etiqueta |  
| `enfocar` | deja en foco **exactamente** los nodos elegidos (no expande vecinos) |  
| `condensar` | funde varios nodos en un macro-concepto, con **linaje restaurable** (badge ■ N) |  
| `criticar` | abre el flujo socrático sobre los nodos elegidos |  
| `actualizar` | muta un nodo existente (fase, etiquetas, descripción) |  
| `delegar` | le pasa el pedido a **Hermes**, que sale a la web con sus herramientas |

En la **barra del nodo** (al seleccionarlo): **Ramificar** (3 ramas) · **Explorar** (viabilidad y riesgos) · **Crítica** (abogado del diablo) · **Socrático** (preguntas que destraban).

**El jardín** (`Panel Jardín`) diagnostica el grafo sin tocarlo: huérfanos, islas, aristas repetidas, nodos sin madurez. Sus hallazgos se convierten en **propuestas** para aprobar. **Reacomodar** ordena el árbol por niveles sin solapamientos.

## 6. Investigación por fases

Dictás **investigá X** y el nodo crece solo: **■ Semilla** (nace al instante) → **■ Fricción** (fuentes reales, un nodo por fuente) → **■ Cápsula** (síntesis y poda) → **■ Hexágono** (cristaliza con su nota).

Corre de fondo: la ventana nunca se congela. Medido: **5 fuentes reales en 87 s**, una del EPA.

## 7. Los motores

- **auto:tarea** (recomendado) elige según lo que pedís, y **el ganador medido va primero**.
- Medición real de esta máquina:

| Motor | Aciertos (5 tareas) | Tiempo | Costo |  
|---|---|---|---|

| `deepseek-r1` (nube, razona) | **\*\*5/5\*\*** | 6,4 s | centavos |  
| `deepseek-chat` (nube) | 4/5 | **\*\*2,7 s\*\*** | centavos |  
| `granite3.3:2b` (local) | 3/5 | 3,7 s | US\$0 |  
| `deepseek-r1:7b` (local) | 4/5 | 14,9 s | US\$0 |

- **\*\*Los locales son la demostración\*\*** de que el diseño no depende de la nube; en una PC con más recursos rinden mejor. La nube es la red de seguridad, no el camino principal.  
- **\*\*Ahorro medido\*\***: los prompts están ordenados para el caché de prefijo del proveedor. Resultado real:  
de **\*\*256 a 1.152\*\*** tokens de entrada servidos desde caché (**\*\*57-58%\*\*** a precio de caché).

## 8. La voz

- **\*\*Entrada\*\***: Speechmatics (token temporal firmado por el backend; la clave nunca sale de ahí).  
- **\*\*Salida\*\***: **\*\*Kokoro local\*\*** — verificado: 104 KB de WAV en 1,9 s.  
- **\*\*Habla selectiva\*\***: sólo avisa hallazgos y contradicciones; cuando actúa, actúa en silencio. El **\*\*mute\*\*** se respeta y persiste.

## 9. El motor profundo

`delegar` corre una pasada completa de Hermes con sus herramientas (búsqueda web incluida) **\*\*sin bloquear\*\*** (0,0 s contra los 215 s de antes) y la respuesta llega sola al panel. Medido: una investigación real de 28 s que devolvió un sensor vigente, su cumplimiento REACH/RoHS y dos alternativas.

## 10. Seguridad y gobernanza

- Toda escritura del agente es **\*\*propuesta\*\*** hasta que la aprobás (`mode: apply` es explícito).  
- El **\*\*núcleo\*\*** no se borra. Las aristas rotas o repetidas se rechazan al escribir.  
- Las **\*\*claves\*\*** viven en `%APPDATA%\com.nodeflow.desktop\nodeflow.config.json` (o en variables de entorno) y nunca vuelven al frontend ni al repositorio.  
- El autoguardado **\*\*no guarda mientras compila\*\***.

## 11. Lo que funciona, lo que es demostración y lo que falta

La parte más útil para nosotros: acá están los puntos ciegos, no las virtudes.

| Función | Estado real |  
|---|---|  
| Crear/conectar/enfocar/condensar/criticar/actualizar | ■ funciona, verificado por API y en la app |  
| Investigación por fases | ■ funciona (87 s, 5 fuentes reales) |

| Formas por estado y mutación | ■ funciona (mide el DOM: sin recortes) |  
| Jardín: diagnóstico y propuestas | ■ funciona (el saneo limpia colgadas **\*\*y\*\*** repetidas) |  
| Voz entrada/salida | ■ funciona (WAV real; el token de Speechmatics se firma) |  
| Delegar a Hermes | ■ funciona sin bloquear |  
| Tavily con texto real de las páginas | ■ la clave **\*\*está puesta y verificada\*\*** (la API devuelve el contenido completo de las páginas: 5.492 y 7.176 caracteres en 1,18 s); falta **\*\*una corrida de investigación entera\*\*** con la clave viva para verla citar |  
| Escalada a la nube | ■ corre, pero los modelos ``-cloud`` del daemon devuelven HTTP 402 (sin créditos) |  
| Caché semántico | ■ falta (hoy hay caché **\*\*exacta\*\***: 20.519 tokens evitados medidos) |  
| Escritura de streaming (partial JSON) | ■ falta: el primer nodo brota al terminar la fase, no antes |  
| Semiótica de aristas (ámbar = contradicción, *\*marching ants\** al dictar) | ■ falta |  
| Contraste fino (chips de 9-10 px del sidebar) | ■ 24 violaciones restantes (peor 2,17) |  
| Autoguardado | ■ se adelanta y commitea cambios que no escribió (mensaje genérico) |  
| Selección múltiple real | ■ el usuario debe probarla una vez antes de grabar (no se puede simular) |

## 12. Cómo mostrarlo (guion de 3 tomas que se ven solas)

1. **\*\*El nodo que investiga\*\***: dictás **\*\*"investigá sensores de humedad de suelo"\*** y el nodo nace, se llena de fuentes y **\*\*cambia de forma\*\*** al subir de fase — sin tocar nada.
2. **\*\*El jardín\*\***: un clic en el diagnóstico y las propuestas aparecen (23 aristas repetidas, nodos huérfanos)  
→ aprobás → la salud del grafo sube.
3. **\*\*La voz\*\***: hablás, la tarjeta de impacto aparece, aprobás, y Kokoro dice el hallazgo.

---

\*NodeFlow · MIT · por TOMAS.WAV\*